

Containerization και Ενορχήστρωση με Docker



Πειραματική σύγκριση αρχιτεκτονικής container και στρατηγικής ενορχήστρωσης

Πλατφόρμες και Αρχιτεκτονικές Νέφους Εξαμηνιαία Εργασία
Θεματική Περιοχή 5: Σύγχρονες Cloud Τεχνολογίες Docker / Containers

Αντώνιος Κασωτάκης A.M. inf2021084

Σεπτέμβριος 2026

1. Εισαγωγή

Η παρούσα εργασία εκπονήθηκε στο πλαίσιο του μαθήματος Πλατφόρμες και Αρχιτεκτονικές Νέφους. Από τις πέντε θεματικές περιοχές που προτείνει η εκφώνηση επιλέχθηκε η 5η, «Σύγχρονες Cloud Τεχνολογίες», και συγκεκριμένα το σκέλος του Docker και των containers· οι άλλες δύο επιλογές της ίδιας ενότητας, το Kubernetes και οι serverless/lambda αρχιτεκτονικές, δεν εξετάζονται στην παρούσα εργασία.

Ένα σημείο πρέπει να διευκρινιστεί εξ αρχής, καθώς επηρεάζει την ερμηνεία όλων των αποτελεσμάτων που ακολουθούν: η εκφώνηση ζητά, ειδικά για αυτή τη θεματική περιοχή, την ανάπτυξη μιας εφαρμογής σε δημόσια πλατφόρμα (AWS, Azure ή Google Cloud).

Η απαίτηση αυτή δεν καλύφθηκε, καθώς δεν υπήρχε διαθέσιμος λογαριασμός κατά την εκπόνηση της εργασίας, και το σύνολο των πειραμάτων που παρουσιάζονται παρακάτω εκτελέστηκε σε έναν τοπικό Docker host.

Όστε να μην παραμείνει εντελώς ανεξέταστο το σκέλος της ενορχήστρωσης, χρησιμοποιήθηκε επιπλέον ένα Docker Swarm ενός κόμβου, το οποίο θέτει σε λειτουργία πραγματικούς μηχανισμούς ενορχήστρωσης χρονοπρογραμματισμό, routing mesh, κλιμάκωση, αυτο-επούλωση χωρίς όμως να εκτείνεται σε πολλαπλά φυσικά μηχανήματα. Πρόκειται για σημαντικό περιορισμό της εργασίας, ο οποίος συζητείται αναλυτικά στην Ενότητα 6.

Η εκφώνηση ζητά την υλοποίηση τουλάχιστον δύο μεθόδων προς σύγκριση. Για την κάλυψη αυτής της απαίτησης με στοιχεία πραγματικά συγκρίσιμα μεταξύ τους, σχεδιάστηκαν δύο ξεχωριστά πειράματα πάνω στην ίδια εφαρμογή:

- **Αρχιτεκτονική:** σύγκριση ενός container που περιλαμβάνει το σύνολο της εφαρμογής και της βάσης δεδομένων της, με έναν σχεδιασμό όπου το καθένα εκτελείται σε ξεχωριστό container. Στην ουσία, εξετάζεται κατά πόσο αξίζει να τηρείται ο κανόνας «μία διεργασία ανά container».
- **Ενορχήστρωση:** η ίδια πολυ-container εφαρμογή, κλιμακωμένη σε τρία αντίγραφα, μία φορά μέσω Docker Compose (με nginx ως reverse proxy για την κατανομή φορτίου) και μία φορά μέσω Docker Swarm, το οποίο διαθέτει δικό του routing mesh χωρίς να απαιτείται εξωτερικός proxy.

Και τα δύο πειράματα χρησιμοποιούν την ίδια εφαρμογή, το ίδιο πραγματικό dataset, και αξιολογούνται με το ίδιο εργαλείο δοκιμής φόρτου και έγχυσης σφαλμάτων, ώστε οι πέντε διαμορφώσεις που τελικά μετρήθηκαν μονολιθικό, Compose 1x, Compose 3x, Swarm 1x, Swarm 3x να είναι πράγματι συγκρίσιμες μεταξύ τους.

2. Θεωρητικό Υπόβαθρο

2.1 Containers και ο κανόνας «μία διεργασία ανά container»

Το Docker συσκευάζει μια εφαρμογή μαζί με τις εξαρτήσεις της βιβλιοθήκες, runtime, ρυθμίσεις σε μια εικόνα, η οποία εκτελείται ως απομονωμένη διεργασία πάνω στον πυρήνα του host, και όχι ως πλήρης εικονική μηχανή με ξεχωριστό λειτουργικό σύστημα. Αυτό αποτελεί και το βασικό πλεονέκτημά του έναντι των εικονικών μηχανών: σημαντικά μικρότερο overhead και ταχύτερη εκκίνηση.

Μια ευρέως αποδεκτή καλή πρακτική στο Docker είναι η αρχή «μία διεργασία ανά container»: μια εφαρμογή ιστού και η βάση δεδομένων της τοποθετούνται σε ξεχωριστά containers, ώστε το καθένα να μπορεί να επανεκκινηθεί, να κλιμακωθεί ή να ενημερωθεί ανεξάρτητα.

Η αντίθετη προσέγγιση, δηλαδή η συγκέντρωση πολλαπλών διεργασιών μακράς διάρκειας μέσα σε ένα μόνο container, θεωρείται γενικά αντι-πρότυπο, παραμένει ωστόσο αρκετά συνηθισμένη στην πράξη, ιδίως σε μικρότερα ή παλαιότερα συστήματα. Αυτό αποτελεί και το αντικείμενο του Πειράματος 1: αν συσκευαστούν όλα τα συστατικά μαζί, ποιο είναι το πραγματικό κόστος;

2.2 Docker Compose

Το Docker Compose περιγράφει δηλωτικά μια πολυ-κοντέινερ εφαρμογή μέσα σε ένα αρχείο YAML: ποιες εικόνες κατασκευάζονται, πώς επικοινωνούν τα containers μεταξύ τους μέσω κοινού δικτύου, ποιες θύρες δημοσιεύονται, καθώς και πολιτικές επανεκκίνησης και όρια πόρων. Είναι σχεδιασμένο να λειτουργεί γύρω από έναν μοναδικό host και όχι σε cluster. Μπορεί να εκτελέσει πολλαπλά αντίγραφα της ίδιας υπηρεσίας, δεν διαθέτει όμως ενσωματωμένο load balancer· η κατανομή κίνησης ανάμεσα σε αντίγραφα που μοιράζονται μία δημοσιευμένη θύρα απαιτεί ένα επιπλέον συστατικό, συνήθως έναν reverse proxy όπως το nginx, όπως εφαρμόζεται στο Πείραμα 2.

2.3 Docker Swarm

Το Docker Swarm είναι ο ενσωματωμένος orchestrator του Docker Engine· ο ίδιος daemon που εκτελεί τις εντολές docker μπορεί να συμμετέχει σε ένα cluster κόμβων και να εκτελεί δηλωτικά «stacks» υπηρεσιών, χωρίς να απαιτείται η εγκατάσταση κάποιου επιπλέον εργαλείου. Δύο χαρακτηριστικά του είναι κεντρικής σημασίας για το Πείραμα 2:

- **Routing mesh:** οποιοσδήποτε κόμβος του cluster δεχτεί κίνηση σε μια δημοσιευμένη θύρα τη δρομολογεί, σε επίπεδο πυρήνα (μέσω IPVS), σε ένα υγιές task της αντίστοιχης υπηρεσίας, ανεξάρτητα από τη θέση του μέσα στο cluster, χωρίς να απαιτείται reverse proxy.
- **Συμφιλίωση** (reconciliation): το Swarm ελέγχει συνεχώς αν ο αριθμός των αντιγράφων που εκτελούνται ταιριάζει με τον επιθυμητό, και σε περίπτωση αστοχίας κάποιου αντικαθιστά αυτόματα το αντίστοιχο task ανεξάρτητα από την κλασική πολιτική επανεκκίνησης που χρησιμοποιούν το docker run ή το Compose.

Το Swarm εκτελέστηκε σε λειτουργία ενός κόμβου, μόνο ως manager και χωρίς workers, με αποτέλεσμα η αντοχή σε αστοχία ολόκληρου μηχανήματος και η δικτύωση μεταξύ κόμβων να μην εξετάζονται στην παρούσα εργασία. Οι μηχανισμοί χρονοπρογραμματισμού, routing mesh και συμφιλίωσης όμως λειτουργούν με τον ίδιο τρόπο ανεξαρτήτως μεγέθους cluster, οπότε τα συμπεράσματα που προκύπτουν παραμένουν έγκυρα.

3. Σύστημα υπό Δοκιμή: το Movie API

Για τις ανάγκες και των δύο πειραμάτων κατασκευάστηκε μια μικρή, ρεαλιστική εφαρμογή REST API, αρκετά ουσιαστική ώστε να παράγει σημαντικό μεικτό φόρτο ανάγνωσης/εγγραφής, αλλά αρκετά απλή ώστε να μη δυσχεραίνει τη σύγκριση.

3.1 Το σύνολο δεδομένων

Ως σύνολο δεδομένων χρησιμοποιήθηκε το πραγματικό dataset *MovieLens «latest-small»* του GroupLens Research (Πανεπιστήμιο της Μινεσότα): 9.742 ταινίες και 100.836 βαθμολογίες από 610 χρήστες. Διατίθεται ελεύθερα από το files.grouplens.org, δεν απαιτεί λογαριασμό ή κλειδί API, και το μέγεθός του επιτρέπει την απευθείας ενσωμάτωσή του μέσα στις εικόνες των containers.

3.2 Το API

Το API υλοποιήθηκε ως εφαρμογή Flask, εξυπηρετούμενη από Gunicorn, με backend το Redis:

Μέθοδος & διαδρομή	Περιγραφή
GET /	Ανακατεύθυνση στο /help.html
GET /help.html	Στατική σελίδα με παραδείγματα curl για κάθε endpoint
GET /health	Έλεγχος ετοιμότητας χρησιμοποιείται από τα health checks των containers και το εργαλείο δοκιμής
GET /stats	Βασικά στατιστικά του dataset
GET /movies/search?q=	Αναζήτηση τίτλων ταινιών, χωρίς διάκριση πεζών/κεφαλαίων
GET /movies/<id>	Μεταδεδομένα ταινίας και τρέχων μέσος όρος βαθμολογίας
POST /movies/<id>/rate	Υποβολή βαθμολογίας· ο μέσος όρος ενημερώνεται ατομικά μέσω ενός Lua script στο Redis

Πίνακας 1. Endpoints του Movie API.

Κάθε απόκριση JSON περιλαμβάνει το hostname του container που την εξυπηρέτησε, στοιχείο χρήσιμο για την επαλήθευση ότι η κίνηση κατανέμεται πράγματι ανάμεσα σε πολλαπλά αντίγραφα.

3.3 Επιλογή του Redis ως backend

Το Redis επιλέχθηκε ως το μοναδικό backend αποθήκευσης και για τις δύο αρχιτεκτονικές, καθώς εκκινεί σε χρόνο μικρότερο του ενός δευτερολέπτου, έχει ελάχιστο αποτύπωμα μνήμης για το συγκεκριμένο μέγεθος dataset, και υποστηρίζει τόσο απλές αναζητήσεις κλειδιού-τιμής όσο και ατομικές λειτουργίες ανάγνωσης-τροποποίησης-εγγραφής, μέσω ενός μικρού Lua script, για την ενημέρωση των βαθμολογιών. Επιλέχθηκε ένα αρκετά ελαφρύ backend ώστε μια βαρύτερη βάση δεδομένων να μην επηρεάζει τις μετρήσεις χρήσης πόρων της Ενότητας 5.

4. Πειραματικός Σχεδιασμός

4.1 Πείραμα 1: Αρχιτεκτονική (μονολιθικό vs πολυ-container)

Κατασκευάστηκαν δύο εικόνες Docker από τον ίδιο κώδικα εφαρμογής:

- **Μονολιθικό** (Dockerfile.monolith): το Redis και η εφαρμογή Flask εκτελούνται μέσα στο ίδιο container. Το entrypoint εκκινεί αρχικά το redis-server στο παρασκήνιο, αναμένει την απόκρισή του σε ping, φορτώνει τα δεδομένα, και στη συνέχεια εκτελεί (exec) το Gunicorn, το οποίο καθίσταται η κύρια διεργασία του container.
- **Πολυ-κοντέινερ** (Dockerfile.app): περιλαμβάνει μόνο την εφαρμογή Flask, η οποία συνδέεται με ξεχωριστό container Redis μέσω του δικτύου του Compose (compose/docker-compose.multi.yml).

Για κάθε διαμόρφωση μετρήθηκαν: το τελικό μέγεθος της εικόνας, ο χρόνος από την εκκίνηση του container έως την πρώτη επιτυχή απόκριση στο /health, η κατανάλωση CPU και μνήμης σε σταθερή κατάσταση (docker stats, μέσος όρος λίγων δευτερολέπτων), το throughput και η καθυστέρηση υπό σταθερό φόρτο εργασίας, καθώς και ο χρόνος ανάκαμψης μετά τον βίαιο τερματισμό του container εφαρμογής εν μέσω κίνησης.

4.2 Πείραμα 2: Ενορχήστρωση (Compose vs Swarm)

Στη συνέχεια, η πολυ-container εφαρμογή κλιμακώθηκε σε τρία αντίγραφα, με δύο διαφορετικές στρατηγικές ενορχήστρωσης:

- Compose με nginx (3 αντίγραφα): τρεις σταθερές υπηρεσίες εφαρμογής (app1, app2, app3) πίσω από έναν nginx που εκτελεί κατανομή φορτίου τύπου round-robin (compose/docker-compose.multi-3x.yml, compose/nginx/nginx.conf) ο τυπικός τρόπος κλιμάκωσης μιας εφαρμογής Compose, δεδομένου ότι το ίδιο το Compose δεν διαθέτει load balancer.
- Swarm (3 αντίγραφα): μία υπηρεσία app (compose/swarm-stack.yml), αναπτυγμένη με docker stack deploy και κλιμακωμένη με docker service scale app=3, με κατανομή φορτίου μέσω του ενσωματωμένου routing mesh, χωρίς κανέναν reverse proxy.

Για κάθε διαμόρφωση μετρήθηκαν: ο χρόνος ανάπτυξης έως την επίτευξη υγιούς κατάστασης, η χρήση πόρων, το throughput και η καθυστέρηση σε 1x και 3x αντίγραφα, καθώς και η συμπεριφορά ανάκαμψης μετά τον τερματισμό ενός backend container.

4.3 Εργαλείο δοκιμής φόρτου και έγχυσης σφαλμάτων

Για τις ανάγκες των πειραμάτων αναπτύχθηκε ένα εργαλείο δοκιμής φόρτου σε Python (bench/load_test.py), το οποίο εκτελεί έναν ρυθμιζόμενο αριθμό ταυτόχρονων νημάτων εργασίας πάνω στο API για καθορισμένη διάρκεια, με μείγμα 80% αναζητήσεων ταινιών και 20% υποβολών βαθμολογιών σε τυχαία επιλεγμένα αναγνωριστικά ταινιών από το dataset.

Κάθε αίτημα καταγράφεται (χρονοσήμανση, καθυστέρηση, επιτυχία ή αποτυχία), παράγοντας:

- Συνολικό throughput και εκατοστημόρια καθυστέρησης (p50/p95/p99).
- Ποσοστό σφαλμάτων ανά δευτερόλεπτο, ώστε μια σύντομη αιχμή αποτυχιών να είναι ορατή και να μην αποκρύπτεται μέσα σε έναν συνολικό μέσο όρο.

- Στις δοκιμές έγχυσης σφαλμάτων, ένα νήμα στο παρασκήνιο τερματίζει συγκεκριμένο container 5 δευτερόλεπτα μετά την έναρξη της δοκιμής, και το εργαλείο υπολογίζει τη μεταβλητή `recovery_time_s`: τον χρόνο από την πρώτη καταγεγραμμένη αποτυχία έως το πρώτο επιτυχημένο αίτημα που ακολουθεί. Η μέτρηση δεν ξεκινά από τη στιγμή του τερματισμού, καθώς ένα αίτημα που είχε ήδη ξεκινήσει λίγο νωρίτερα μπορεί να ολοκληρωθεί επιτυχώς αμέσως μετά, κάτι που θα υποεκτιμούσε τον πραγματικό χρόνο διακοπής.

Το όριο χρόνου ανά αίτημα ορίστηκε σκόπιμα μικρό, στο 1,5 δευτερόλεπτο. Στην αρχική έκδοση του εργαλείου το όριο ήταν 5 δευτερόλεπτα, με αποτέλεσμα, μόλις ένα backend σταματούσε να λειτουργεί, τα νήματα εργασίας να παραμένουν μπλοκαρισμένα σε ένα εκκρεμές αίτημα για ολόκληρο αυτό το διάστημα αντί να επαναλαμβάνουν την προσπάθεια στερώντας ακριβώς τη λεπτομέρεια που ήταν απαραίτητη για την παρατήρηση της ανάκαμψης.

4.4 Περιβάλλον δοκιμών

Στοιχείο	Τιμή
Host	Ένας τοπικός υπολογιστής 4 vCPUs, 34 GB RAM
Λειτουργικό σύστημα	Debian 13, πυρήνας 7.1.5
Docker Engine	29.8.0, ένας κόμβος (χωρίς Swarm workers)
Εργαλείο δοκιμής φόρτου	Εκτελέστηκε στον ίδιο host με τα containers υπό δοκιμή
Dataset	MovieLens «latest-small» (9.742 ταινίες, 100.836 βαθμολογίες)

Πίνακας 2. Πειραματικό περιβάλλον.

Η εκτέλεση του εργαλείου δοκιμής φόρτου στο ίδιο μηχάνημα με τα containers υπό δοκιμή σημαίνει ότι client και server μοιράζονταν τους ίδιους 4 πυρήνες CPU.

Οι απόλυτες τιμές throughput πρέπει συνεπώς να ερμηνεύονται συγκριτικά, ανάμεσα στις διαμορφώσεις που δοκιμάστηκαν, και όχι ως ένδειξη πραγματικής χωρητικότητας παραγωγικού περιβάλλοντος.

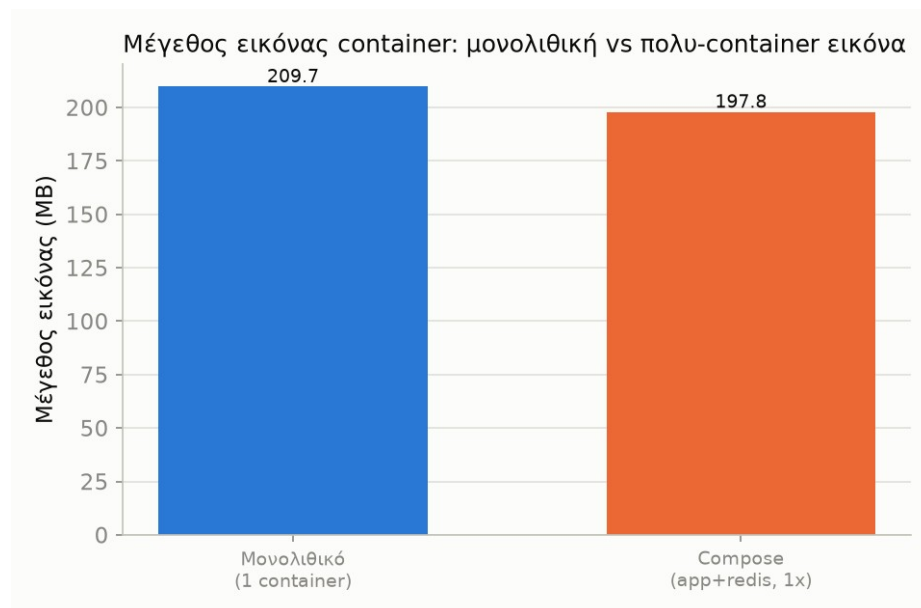
5. Αποτελέσματα και Ανάλυση

Και οι πέντε διαμορφώσεις μονολιθικό, Compose (1 αντίγραφο), Compose (3 αντίγραφα + nginx), Swarm (1 αντίγραφο), Swarm (3 αντίγραφα) δοκιμάστηκαν με το εργαλείο της Ενότητας 4.3. Ο Πίνακας 3 συγκεντρώνει το σύνολο των μετρήσεων· η ανάλυση που ακολουθεί εξετάζει κάθε διάγραμμα ξεχωριστά.

Διαμόρφωση	Εκκίνηση (s)	Εικόνα (MB)	Throughput (αιτ./s)	p95 (ms)	Μνήμη (MB)	Σφάλμα %	Ανάκαμψη (s)
Μονολιθικό	1,02	209,7	1182,8	48,3	62,1	8,15%	καμία ανάκαμψη
Compose (1x)	1,00	197,8	1206,2	45,5	65,9	7,30%	καμία ανάκαμψη
Compose (3x+nginx)	0,63	197,8 *	1438,6	36,9	172,0	0,02%	0,07
Swarm (1 αντίγραφο)	7,54	197,8 *	2098,0	13,5	53,3	0,68%	9,23
Swarm (3 αντίγραφα)	+	197,8 *	1601,0	38,9	161,3	0,12%	0,05

Πίνακας 3. Πλήρης σύνοψη αποτελεσμάτων. * Οι διαμορφώσεις Compose 3x και Swarm επαναχρησιμοποιούν την ίδια εικόνα εφαρμογής πολλαπλών containers που μετρήθηκε στη γραμμή «Compose (1x)». + Η κλιμάκωση του Swarm από 1 σε 3 αντίγραφα χρειάστηκε περίπου 20–35 δευτερόλεπτα για να συγκλίνει (παρατηρήθηκε μέσω `docker service ps`, χωρίς να καταγραφεί ως ακριβής μετρική) πιο αργά από μια νέα ανάπτυξη, καθώς πρόκειται για κυλιόμενη ενημέρωση και όχι εκκίνηση από το μηδέν.

5.1 Μέγεθος εικόνας

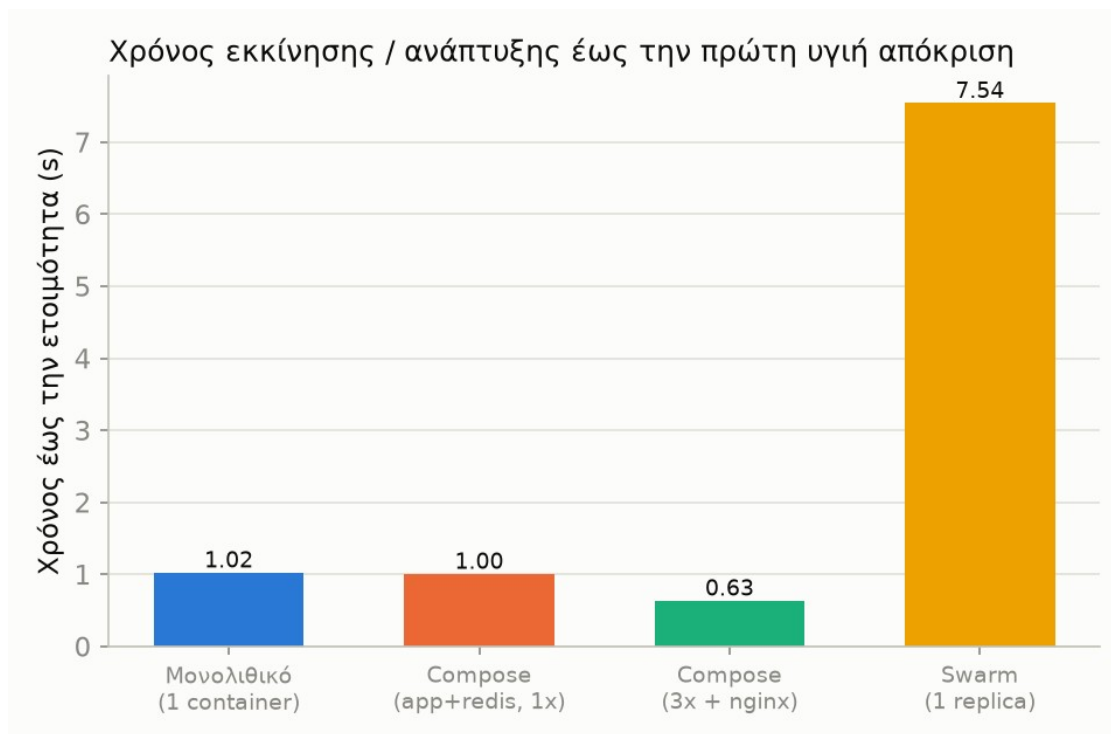


Διάγραμμα 1. Μέγεθος εικόνας container, μονολιθικό vs εικόνα εφαρμογής πολλαπλών containers.

Η μονολιθική εικόνα προέκυψε περίπου 6% μεγαλύτερη σε σχέση με την πολυ-container εικόνα (209,7 MB έναντι 197,8 MB), λόγω των επιπλέον πακέτων apt (redis-server και των εξαρτήσεών του) που ενσωματώνονται σε αυτήν.

Πρόκειται για μέτρια διαφορά, η οποία δεν αποτελεί από μόνη της ισχυρό επιχείρημα προς καμία κατεύθυνση, προστίθεται όμως στα πιο ουσιαστικά λειτουργικά μειονεκτήματα που παρουσιάζονται στην Ενότητα 5.4.

5.2 Χρόνος εκκίνησης και ανάπτυξης



Διάγραμμα 2. Χρόνος από την εκκίνηση του container/stack έως την πρώτη επιτυχή απόκριση υγείας.

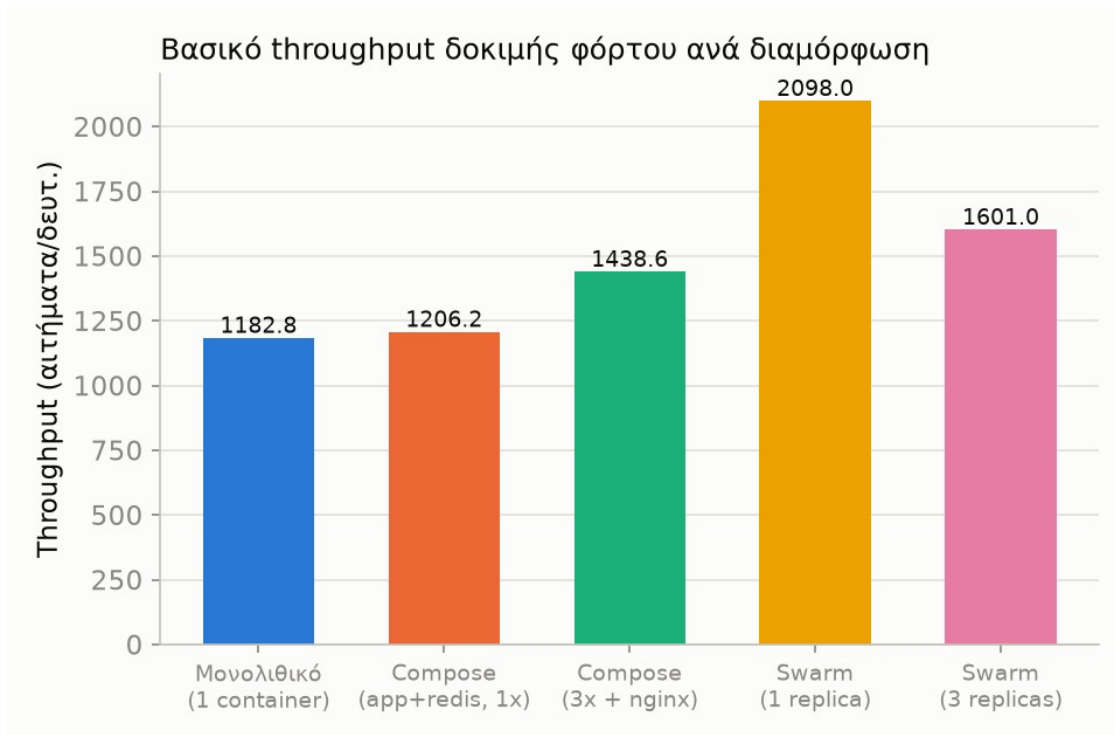
Το μονολιθικό και το Compose ενός αντιγράφου εκκινούν σε παρόμοιο χρόνο (περίπου 1 δευτερόλεπτο), αναμενόμενο δεδομένου ότι πρόκειται και στις δύο περιπτώσεις για ένα μοναδικό container που εκτελεί την ίδια εφαρμογή Unicorn· το επιπλέον βήμα εκκίνησης του τοπικού Redis στο μονολιθικό προσθέτει αμελητέο χρόνο.

Αξιοσημείωτο είναι ότι η ανάπτυξη Compose με 3 αντίγραφα εμφανίζει τον μικρότερο χρόνο έως την πρώτη υγιή απόκριση (0,63s): καθώς τα τρία containers εκκινούν παράλληλα, το health check αρκεί να απαντηθεί από ένα μόνο από αυτά.

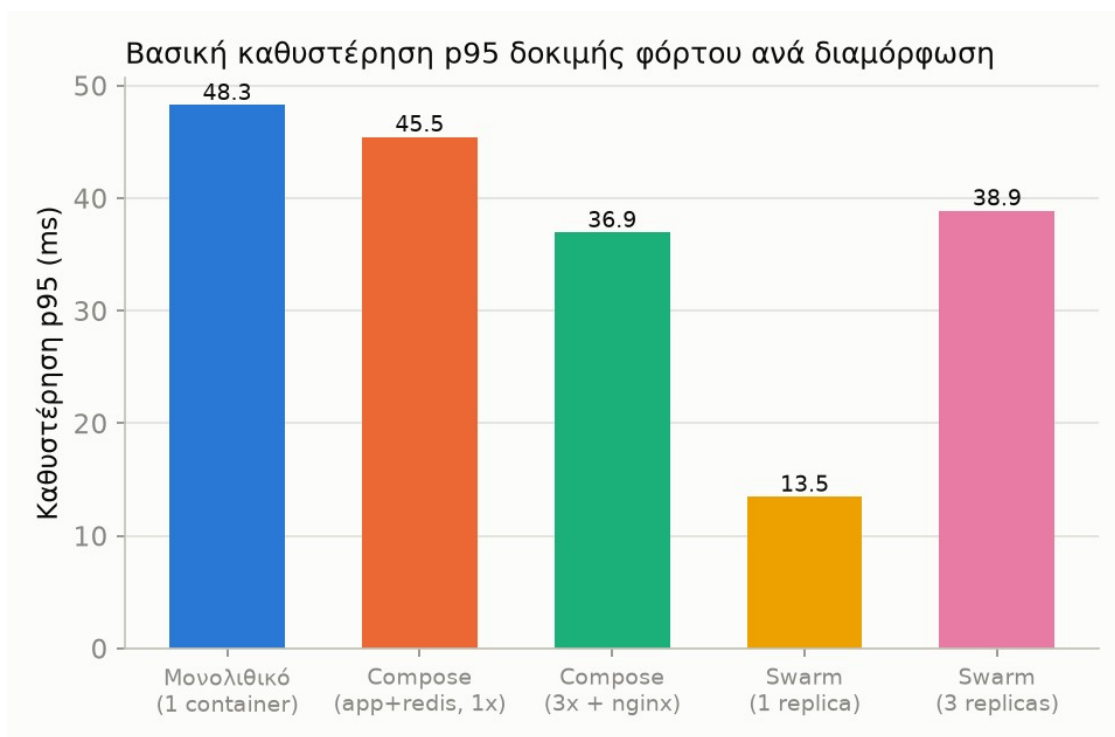
Το Swarm, αντίθετα, παρουσιάζει σημαντικά μεγαλύτερο χρόνο (7,54s), καθώς απαιτείται αρχικά η δημιουργία ενός overlay δικτύου (βασισμένου σε VXLAN, σχεδιασμένου για κίνηση μεταξύ πολλαπλών κόμβων) και η ολοκλήρωση της διαδικασίας χρονοπρογραμματισμού πριν από την εκκίνηση του container.

Πρόκειται για κόστος που καταβάλλεται μία φορά κατά την εκκίνηση, με αντάλλαγμα τις δυνατότητες routing mesh και συμφιλίωσης που παρουσιάζονται παρακάτω.

5.3 Βασικό throughput και καθυστέρηση



Διάγραμμα 3. Throughput δοκιμής φόρτου ανά διαμόρφωση (20–30 ταυτόχρονοι πελάτες, μείγμα ανάγνωσης/εγγραφής 80/20, 15s).



Διάγραμμα 4. Καθυστέρηση p95 ανά διαμόρφωση, στην ίδια δοκιμή φόρτου.

Το πλέον αξιοσημείωτο εύρημα της ενότητας είναι ότι η διαμόρφωση Swarm με 1 αντίγραφο εμφανίζει το υψηλότερο throughput από τις πέντε διαμορφώσεις συνολικά (2098 αιτ./s, 13,5ms p95), ξεπερνώντας ακόμη και το Swarm με 3 αντίγραφα. Το εύρημα αυτό φαίνεται αντιφατικό εκ πρώτης όψεως, εξηγείται όμως από το γεγονός ότι όλες οι

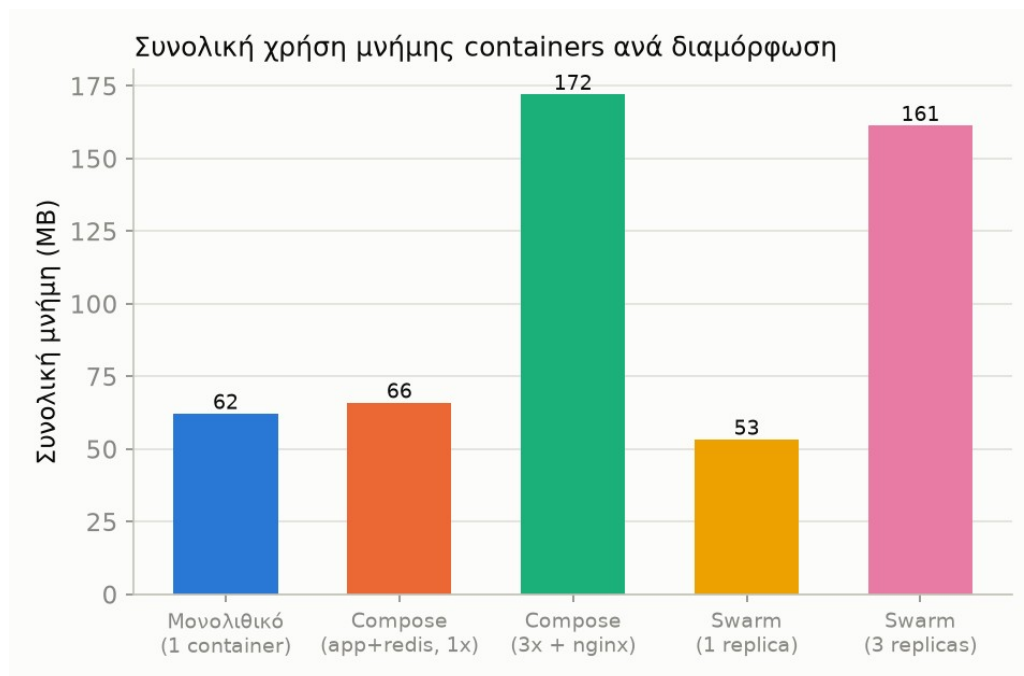
δοκιμές εκτελέστηκαν σε ένα μηχάνημα με 4 πυρήνες: το routing mesh του Swarm λειτουργεί σε επίπεδο πυρήνα (μέσω IPVS) και δεν προσθέτει ουσιαστική επιβάρυνση για ένα μοναδικό task, ενώ στην κλιμάκωση σε 3 αντίγραφα, τρεις διεργασίες Gunicorn και το ίδιο το routing mesh ανταγωνίζονται πλέον για τους ίδιους 4 πυρήνες επεξεργαστή, μοιράζοντας παράλληλα ένα κοινό instance Redis.

Οι διαμορφώσεις Compose παρουσιάζουν μικρότερα, πιο αναμενόμενα κέρδη κατά την κλιμάκωση (1206 → 1439 αιτ./s), καθώς η επιβάρυνση του nginx ως L7 proxy παραμένει σχετικά σταθερή ανά αίτημα, ανεξαρτήτως του αριθμού αντιγράφων πίσω από αυτόν.

Το εύρημα αυτό δεν πρέπει να ερμηνευθεί ως ένδειξη ότι η απόδοση του Swarm υποβαθμίζεται με την κλιμάκωση, αλλά ως συνέπεια του πειραματικού περιβάλλοντος: η οριζόντια κλιμάκωση αποδίδει οφέλη μόνο όταν υπάρχει διαθέσιμο επιπλέον υλικό, και μια δίκαιη σύγκριση θα απαιτούσε περισσότερους από έναν κόμβους ακριβώς το βήμα ανάπτυξης σε cloud που δεν καλύφθηκε στην παρούσα εργασία (βλ. Ενότητα 6.1).

Η καθυστέρηση επιβεβαιώνει την ίδια εικόνα: η δρομολόγηση του Swarm σε επίπεδο πυρήνα (13,5ms p95, 1 αντίγραφο) είναι περίπου 2,5 φορές ταχύτερη από τον nginx σε επίπεδο εφαρμογής (36,9ms p95, 3 αντίγραφα), παρότι το δεύτερο εξυπηρετεί τριπλάσια χωρητικότητα backend.

5.4 Χρήση πόρων

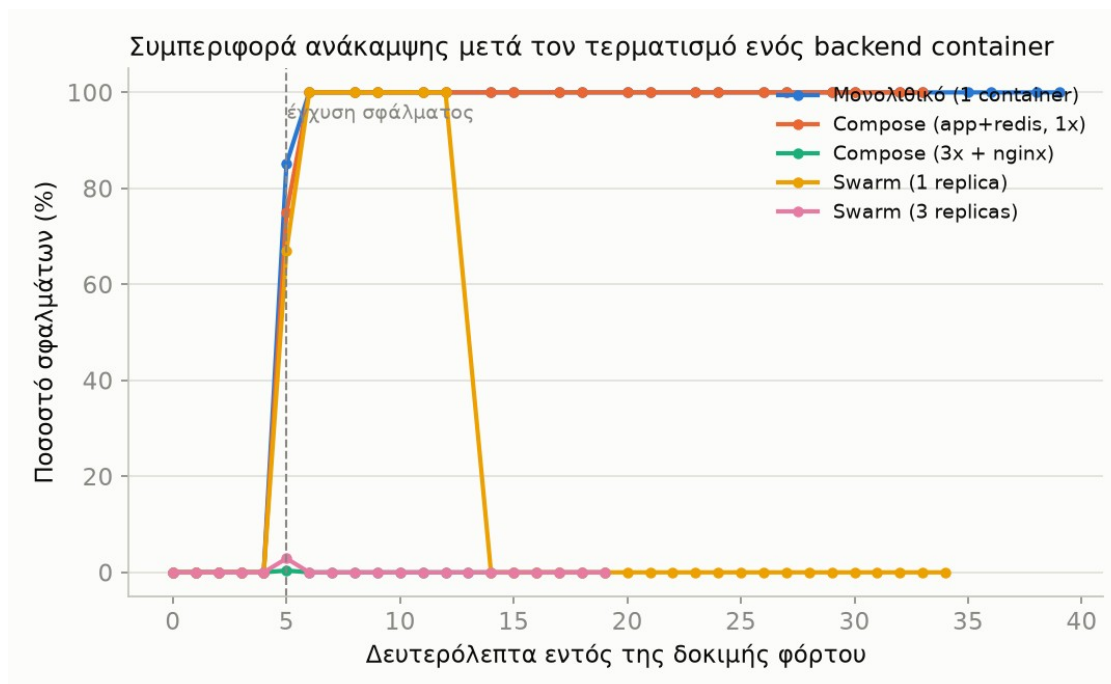


Διάγραμμα 5. Συνολική χρήση μνήμης σε όλα τα containers της διαμόρφωσης.

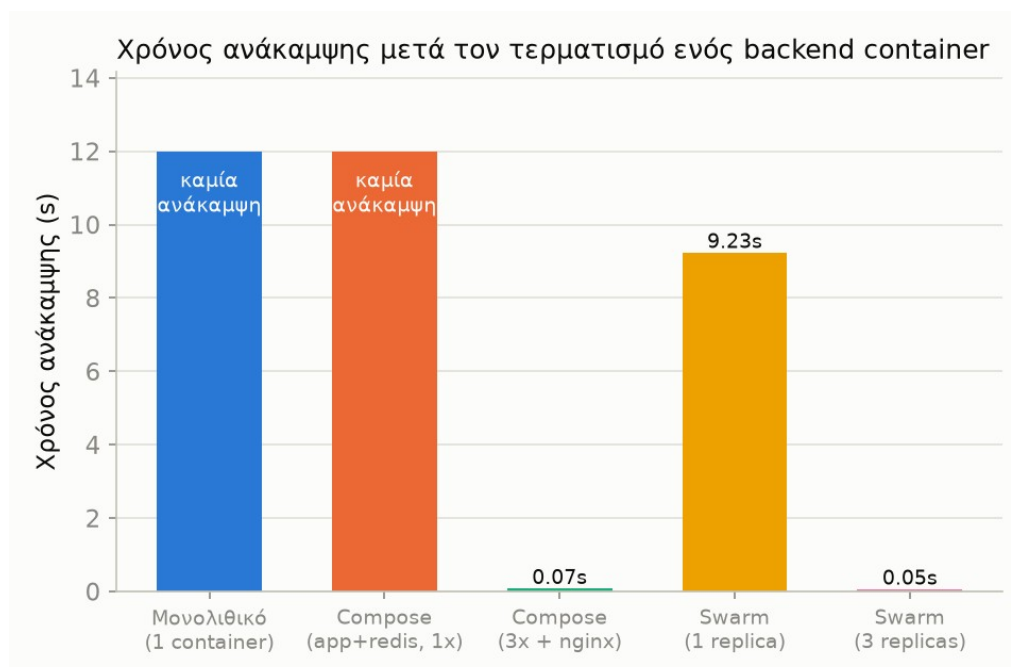
Η κατανάλωση μνήμης κλιμακώνεται σχεδόν γραμμικά με τον αριθμό των αντιγράφων, όπως αναμενόταν (τρεις διεργασίες backend στα 53–66 MB η καθεμία), με μια μικρή σταθερή προσαύξηση ανάλογα με τον μηχανισμό κατανομής φορτίου: το nginx προσθέτει περίπου 11 MB πάνω από το τριπλάσιο αποτύπωμα app+redis, ενώ το routing mesh του Swarm, μη απαιτώντας proxy σε επίπεδο

εφαρμογής, οδηγεί σε ελαφρώς μικρότερη συνολική κατανάλωση (161,3 MB έναντι 172,0 MB), παρά το γεγονός ότι εξυπηρετεί παρόμοιο όγκο κίνησης.

5.5 Ανθεκτικότητα σε σφάλματα το κεντρικό εύρημα



Διάγραμμα 6. Ποσοστό σφαλμάτων στον χρόνο, μετά τον τερματισμό ενός backend container 5 δευτερόλεπτα μέσα σε μια ενεργή δοκιμή φόρτου.



Διάγραμμα 7. Χρόνος έως το πρώτο επιτυχημένο αίτημα μετά την πρώτη καταγεγραμμένη αποτυχία (βλ. Ενότητα 4.3 για τον ακριβή ορισμό).

Πρόκειται για το σημαντικότερο αποτέλεσμα της εργασίας, το οποίο διακρίνεται σε δύο επιμέρους ευρήματα.

5.5.1 Πλεονασμός και ορθά ρυθμισμένος load balancer οδηγούν σε σχεδόν ακαριαία ανάκαμψη

Και οι δύο διαμορφώσεις με 3 αντίγραφα ανακάμπτουν σε χρόνο μικρότερο του ενός δεκάτου του δευτερολέπτου (Compose+nginx: 0,07s· Swarm: 0,05s), με αμελητέο συνολικό ποσοστό σφαλμάτων (0,02% και 0,12% αντίστοιχα): τα δύο εναπομείναντα αντίγραφα απορροφούν το φορτίο μέχρι την αντικατάσταση του container που τερματίστηκε.

Το αποτέλεσμα αυτό στο Compose δεν προέκυψε αυτόματα, καθώς απαιτήθηκε ειδική ρύθμιση της συμπεριφοράς failover του nginx (βλ. Ενότητα 6.3). Το Swarm πέτυχε το ίδιο αποτέλεσμα χωρίς καμία πρόσθετη ρύθμιση, καθώς το routing mesh εντοπίζει αυτόματα ένα ανενεργό task και διακόπτει την αποστολή κίνησης προς αυτό, ως εγγενές χαρακτηριστικό της λειτουργίας του.

5.5.2 Χωρίς πλεονασμό δεν παρατηρείται αυτόματη ανάκαμψη

Το μονολιθικό και το Compose ενός αντιγράφου δεν παρουσίασαν καμία ανάκαμψη εντός του παραθύρου δοκιμής (έως 40 δευτερόλεπτα)· κάθε αίτημα απέτυχε από τη στιγμή του τερματισμού του container έως το τέλος της δοκιμής.

Η περαιτέρω διερεύνηση του φαινομένου (Ενότητα 6.3) το συνέδεσε με ζήτημα ειδικό για τη συγκεκριμένη εγκατάσταση Docker Engine, σχετικό με την κλασική πολιτική επανεκκίνησης, το οποίο επιβεβαιώθηκε ανεξάρτητα από τον κώδικα της εργασίας μέσω μιας απλής δοκιμής (`docker run --restart always alpine`).

Η διαμόρφωση Swarm ενός αντιγράφου, αντίθετα, ανακάμπτει σε περίπου 9,2 δευτερόλεπτα, καθώς οι υπηρεσίες Swarm επιτηρούνται από εντελώς διαφορετικό μηχανισμό συνεχή συμφιλίωση επιθυμητού με πραγματικό αριθμό tasks ανεξάρτητο από την πολιτική επανεκκίνησης ανά container.

Το εύρημα αυτό αποτελεί το σημαντικότερο συμπέρασμα της εργασίας: δεν υποδεικνύει απλώς ότι το Swarm διαθέτει καλύτερες προεπιλογές σε σχέση με το Compose, αλλά ότι, στο συγκεκριμένο περιβάλλον δοκιμών, ένα μεμονωμένο container δεν διέθετε κανέναν λειτουργικό μηχανισμό αυτο-επούλωσης, ενώ μια υπηρεσία υπό orchestrator διέθετε, ακριβώς επειδή στηρίζεται σε διαφορετικό υποκείμενο μηχανισμό. Πρόκειται για ουσιαστικό επιχείρημα υπέρ της εκτέλεσης κάθε φόρτου εργασίας παραγωγής κάτω από κάποιον orchestrator, ανεξάρτητα από την ανάγκη οριζόντιας κλιμάκωσης.

6. Συζήτηση και Περιορισμοί

6.1 Απουσία ανάπτυξης σε δημόσιο cloud

Όπως αναφέρθηκε στην Εισαγωγή, η ελάχιστη απαίτηση της εκφώνησης για τη συγκεκριμένη θεματική περιοχή ήταν η ανάπτυξη της demo εφαρμογής σε δημόσια cloud πλατφόρμα, κάτι που δεν πραγματοποιήθηκε λόγω έλλειψης διαθέσιμου λογαριασμού. Το σύνολο των αποτελεσμάτων της Ενότητας 5 προέρχεται από έναν μοναδικό τοπικό Docker host.

Η πιο άμεση συνέπεια αυτού του περιορισμού εντοπίζεται στην Ενότητα 5.3: η οριζόντια κλιμάκωση δεν εμφάνισε όφελος στο throughput, καθώς δεν υπήρχε διαθέσιμο επιπλέον υλικό για να το απορροφήσει. Μια πραγματική ανάπτυξη σε πολλαπλούς κόμβους για παράδειγμα ένα Swarm cluster τριών κόμβων, ή το αντίστοιχο σε managed Kubernetes αναμένεται ευλόγως να αναδείξει οφέλη που ένα μοναδικό μηχάνημα δεν μπορεί να καταδείξει.

6.2 Swarm ενός κόμβου

Σχετικά με το παραπάνω, το Swarm εκτελέστηκε με έναν κόμβο manager και χωρίς workers. Οι μηχανισμοί χρονοπρογραμματισμού, routing mesh και συμφιλίωσης που εξετάστηκαν λειτουργούν με τον ίδιο τρόπο ανεξαρτήτως μεγέθους cluster, ωστόσο η πραγματική αντοχή σε αστοχία κόμβου και η συμπεριφορά δικτύου μεταξύ κόμβων δεν δοκιμάστηκαν.

6.3 Δύο ζητήματα μεθοδολογίας που εντοπίστηκαν και διορθώθηκαν

Κατά τη συλλογή των αποτελεσμάτων έγχυσης σφαλμάτων εντοπίστηκαν δύο ζητήματα μεθοδολογίας, τα οποία καταγράφονται εδώ καθώς επηρεάζουν ουσιαστικά την ερμηνεία των αριθμών ανάκαμψης της Ενότητας 5.5:

- Ρύθμιση failover του nginx. Η αρχική έκδοση του `compose/nginx/nginx.conf` χρησιμοποιούσε τις προεπιλεγμένες ρυθμίσεις του nginx, οι οποίες περιλαμβάνουν όριο χρόνου σύνδεσης 60 δευτερολέπτων. Χειροκίνητη δοκιμή με επαναλαμβανόμενα αιτήματα curl, ενώ ένα backend είχε τερματιστεί, έδειξε ότι περίπου ένα στα τρία αιτήματα αυτά που δρομολογούνταν προς το ανενεργό container παρέμεναν σε αναμονή για αρκετά δευτερόλεπτα αντί να γίνεται άμεσο failover. Το ζήτημα διορθώθηκε με τη ρητή προσθήκη των οδηγιών `proxy_connect_timeout`, `proxy_next_upstream` και `max_fails/fail_timeout`, μετά την οποία το failover έγινε σχεδόν ακαριαίο. Τα αποτελέσματα της Ενότητας 5.5 περιλαμβάνουν ήδη αυτή τη διόρθωση· το γεγονός ότι το Compose απαιτήσε αυτή τη ρύθμιση, ενώ το Swarm όχι, αποτελεί και το ίδιο μέρος της σύγκρισης.
- Ορισμός του χρόνου ανάκαμψης. Η αρχική έκδοση του υπολογισμού μετρούσε το διάστημα από τον τερματισμό του container έως το πρώτο επιτυχημένο αίτημα. Η προσέγγιση αυτή υπερεκτιμούσε την ταχύτητα ανάκαμψης, καθώς ένα αίτημα που βρισκόταν ήδη σε εξέλιξη τη στιγμή του τερματισμού μπορούσε να ολοκληρωθεί επιτυχώς αμέσως μετά, χωρίς αυτό να αποδεικνύει ότι το σύστημα είχε πράγματι ανακάμψει. Η μετρική αναθεωρήθηκε ώστε να υπολογίζει τον χρόνο από την πρώτη καταγεγραμμένη αποτυχία έως την πρώτη

επιτυχία που ακολουθεί, απαιτώντας δηλαδή να έχει παρατηρηθεί πραγματική αποτυχία πριν η υπηρεσία θεωρηθεί ότι ανέκαμψε.

Και τα δύο ζητήματα εντοπίστηκαν μέσω λογικού ελέγχου μη ρεαλιστικών τιμών ένα σταθερό ποσοστό σφαλμάτων περίπου 33% που δεν βελτιωνόταν ποτέ, και έναν υπολογιζόμενο «χρόνο ανάκαμψης» μικρότερο του ενός χιλιοστού του δευτερολέπτου για container χωρίς κανέναν πλεονασμό και διορθώθηκαν πριν από τη συλλογή των τελικών αποτελεσμάτων που παρουσιάζονται στην Ενότητα 5.

6.4 Μια απρόβλεπτη συμπεριφορά του Docker Engine

Το εύρημα ότι το μονολιθικό και το Compose ενός αντιγράφου δεν ανακάμπτουν ποτέ (Ενότητα 5.5.2) επαληθεύτηκε ανεξάρτητα από τον κώδικα της εργασίας: μια απλή δοκιμή (`docker run -d --restart always alpine sleep 3600`, τερματισμένη με `docker kill`) δεν οδήγησε ποτέ σε επανεκκίνηση στην εγκατάσταση Docker Engine 29.8.0 του συγκεκριμένου μηχανήματος, όπως επιβεβαιώθηκε μέσω `docker inspect` (RestartCount ίσο με 0). Πρόκειται για απρόβλεπτη συμπεριφορά, καθώς οι πολιτικές επανεκκίνησης αποτελούν βασική και καλά τεκμηριωμένη λειτουργία του Docker· αναφέρεται εδώ ως ιδιαιτερότητα της συγκεκριμένης εγκατάστασης και όχι ως γενικό χαρακτηριστικό του Docker ή των εικόνων που κατασκευάστηκαν για την εργασία. Δεν επηρεάζει την εγκυρότητα της σύγκρισης μεταξύ αρχιτεκτονικών, καθώς επηρέασε εξίσου το μονολιθικό και το πολυ-container με 1 αντίγραφο· αντίθετα, ενισχύει το συμπέρασμα της Ενότητας 5.5.2, υποδεικνύοντας ότι το ζήτημα εντοπίζεται στον υποκείμενο μηχανισμό (πολιτική επανεκκίνησης έναντι συμφιλίωσης orchestrator) και όχι σε κάτι ειδικό για τη συγκεκριμένη εφαρμογή.

6.5 Κοινό μηχάνημα για client και server

Το εργαλείο δοκιμής φόρτου εκτελέστηκε στο ίδιο μηχάνημα με τα containers υπό δοκιμή (4 πυρήνες), με αποτέλεσμα ο χρονοπρογραμματισμός νημάτων στην πλευρά του client και η επεξεργασία αιτημάτων στην πλευρά του server να ανταγωνίζονται για τους ίδιους πόρους επεξεργαστή. Οι απόλυτες τιμές throughput και καθυστέρησης πρέπει συνεπώς να ερμηνεύονται συγκριτικά, ανάμεσα στις πέντε διαμορφώσεις που δοκιμάστηκαν, και όχι ως αντιπροσωπευτικές της απόδοσης έναντι πραγματικών πελατών δικτύου σε αποκλειστικό υλικό.

7. Συμπεράσματα

Στην παρούσα εργασία τέθηκαν δύο ερωτήματα, στα οποία τα δύο πειράματα έδωσαν σαφή απάντηση.

Ως προς την αρχιτεκτονική, στην κλίμακα που εξετάστηκε, ο σχεδιασμός με ξεχωριστά containers δεν παρουσιάζει ουσιαστικό μειονέκτημα έναντι του μονολιθικού ελαφρώς μικρότερη εικόνα, στατιστικά ισοδύναμη βασική απόδοση και προσφέρει ένα σαφές πλεονέκτημα: η εφαρμογή και η βάση δεδομένων της μπορούν να επανεκκινηθούν, να ενημερωθούν και, όπως προκύπτει από το Πείραμα 2, να κλιμακωθούν ανεξάρτητα η μία από την άλλη. Δεν προέκυψε κανένα επιχείρημα απόδοσης υπέρ της συγκέντρωσης πολλαπλών διεργασιών σε ένα container.

Ως προς την ενορχήστρωση, το Docker Swarm προσφέρει ταχύ και ισορροπημένο failover, καθώς και οριζόντια κλιμάκωση χωρίς καμία πρόσθετη ρύθμιση, με κόστος σημαντικά αργότερη ψυχρή εκκίνηση (περίπου 7,5s έναντι λιγότερο από 1s) και, σε ένα μοναδικό μηχάνημα, καμία υπεροχή στο throughput έναντι μιας καλά ρυθμισμένης ανάπτυξης Compose.

Το Compose μπορεί να προσεγγίσει την ταχύτητα failover του Swarm μετά την κλιμάκωση, αυτό όμως προϋποθέτει τη χειροκίνητη ρύθμιση του ορίου χρόνου και του ελέγχου υγείας του reverse proxy ρύθμιση που παραλείπεται εύκολα, όπως συνέβη και στην αρχική έκδοση της παρούσας εργασίας, και την οποία το Swarm δεν χρειάζεται.

Το πλέον αξιοσημείωτο εύρημα είναι ότι ένα μεμονωμένο container ή μια μη ενορχηστρωμένη υπηρεσία Compose δεν διέθεταν καμία λειτουργική αυτόματη ανάκαμψη στη συγκεκριμένη εγκατάσταση Docker, ενώ ένα μοναδικό αντίγραφο υπό orchestrator ανέκαμψε σε περίπου εννέα δευτερόλεπτα, μέσω εντελώς ανεξάρτητου μηχανισμού.

Η σύσταση που προκύπτει από τα δεδομένα της εργασίας είναι η εκτέλεση υπηρεσιών κάτω από κάποιον orchestrator Swarm, ή σε μεγαλύτερη κλίμακα Kubernetes αντί ως μεμονωμένα containers ή απλές υπηρεσίες Compose, για κάθε ανάπτυξη πέραν του τοπικού περιβάλλοντος, καθώς η αυτο-επούλωση και η ισορροπημένη κλιμάκωση αποτελούν ιδιότητες του επιπέδου ενορχήστρωσης και όχι των containers γενικότερα· όπως προκύπτει από την Ενότητα 6.4, δεν μπορούν μάλιστα να θεωρούνται δεδομένες ούτε από την ίδια την πολιτική επανεκκίνησης του Docker.

Ο βασικός περιορισμός της παρούσας εργασίας, αναγνωρισμένος εξαρχής, είναι η εκτέλεση του συνόλου των πειραμάτων σε τοπικό μηχάνημα και όχι σε δημόσια cloud πλατφόρμα, όπως προέβλεπε η εκφώνηση για τη συγκεκριμένη θεματική περιοχή. Τα αποτελέσματα throughput ειδικότερα θα πρέπει να επαναξιολογηθούν σε πραγματική υποδομή πολλαπλών κόμβων πριν εξαχθούν συμπεράσματα σχετικά με τη χωρητικότητα σε παραγωγικό περιβάλλον.

8. Αναφορές

- Docker Inc. Τεκμηρίωση Docker Engine. <https://docs.docker.com/engine/>
- Docker Inc. Τεκμηρίωση Docker Compose. <https://docs.docker.com/compose/>
- Docker Inc. Επισκόπηση λειτουργίας Swarm και routing mesh. <https://docs.docker.com/engine/swarm/>
- Τεκμηρίωση nginx ngx_http_proxy_module (proxy_connect_timeout, proxy_next_upstream) και ngx_http_upstream_module (max_fails, fail_timeout). <https://nginx.org/en/docs/>
- F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4, Article 19. GroupLens Research, Πανεπιστήμιο της Μινεσότα. <https://grouplens.org/datasets/movielens/>
- Πηγαίος κώδικας, Dockerfiles, αρχεία Compose/Swarm, εργαλείο δοκιμών και ακατέργαστα αποτελέσματα: βλέπε το συνοδευτικό αποθετήριο GitHub.